Project Report: xv6 SYSTEM CALL IMPLEMENTATION

Group Name : Random (G26)
Course: Operating Systems Lab (CSC211)

---

1. Project Overview

This project focuses on extending the xv6 operating system by implementing custom system calls and analysing their behaviour at the kernel level. xv6 is a simple Unix-like teaching OS used to understand operating system concepts in a minimal and controlled environment.

The primary goal of this project is to bridge the gap between user-space applications and kernel-level operations by implementing and testing system calls that handle:

- Process information retrieval
- Inter-process communication (IPC)
- Thread-like execution using process duplication
- Synchronization using locks

Through this project, we explore how user programs interact with kernel functions via system call interfaces.

---

2. Objectives

The main objectives of this project are:

- To understand the internal structure of xv6 system calls
- To implement new custom system calls in the kernel
- To modify kernel code safely without breaking system integrity
- To demonstrate system call usage through user-level programs
- To understand interaction between user space and kernel space
- To explore concepts like IPC, synchronization, and process management

---

3. System Call Architecture in xv6

In xv6, system calls follow a well-defined pipeline:

```
User Program → user.h → usys.pl → syscall.c → sysproc.c → Kernel
```

Key Components:

- **user.h** → Declares system call prototypes for user programs
- **usys.pl** → Generates system call stubs (user entry points)
- **syscall.c** → Dispatches system calls using syscall table
- **sysproc.c** → Contains implementation of process-related system calls
- **kernel/proc.c** → Handles process-level operations

This architecture ensures a secure boundary between user space and kernel space.

---

4. Implemented Functionalities

4.1 getppid() – Process Information System Call

Purpose

Returns the Parent Process ID (PPID) of the currently running process.

Working

- Uses myproc() to access current process structure
- Retrieves parent->pid field
- Returns PPID to user program

Use Case

Helps understand process hierarchy in xv6.

4.2 sendmsg() & recvmsg() – Inter Process Communication (IPC)

Purpose

Implements simple message passing between parent and child processes.

Working

- Parent writes message into shared kernel buffer
- Child reads message using recvmsg()
- Ensures controlled communication between processes

Features

- One-way communication (parent to child)
- Simple buffer-based IPC

4.3 clone() – Thread-like Execution

Purpose

Creates a lightweight execution unit similar to threads.

Working

- Based on fork() system call
- Shares or duplicates process context
- Executes code concurrently

Observation

Simulates basic multithreading behaviour in xv6.

4.4 mylock() & myunlock() – Synchronization Mechanism

Purpose

Implements mutual exclusion to prevent race conditions.

Working

- Uses atomic test-and-set mechanism
- Ensures only one process enters critical section
- Releases lock using myunlock()

Use Case

Protect shared resources like counters.

---

5. Implementation Details

5.1 Kernel-Level Modifications

File	Modification
proc.h	Added message buffer and lock variables
sysproc.c	Implemented custom system calls

syscall.c	Registered syscall handlers
syscall.h	Assigned syscall numbers

5.2 User-Level Modifications

File	Purpose
user.h	Declared system call prototypes
usys.pl	Generated system call stubs
test.c	Created test program for demonstration

---

6. Build & Execution Steps

Compile xv6:

make clean

make qemu

Run test program inside xv6:

test

---

7. Challenges Faced

- Handling naming conflicts with kernel functions (e.g., lock)
- Missing syscall bindings in usys.pl
- Debugging kernel crashes due to incorrect syscall mapping
- Understanding kernel-user boundary communication
- Ensuring safe memory access in IPC implementation

8.Program Output

The custom system calls were tested using a user program (test.c) inside xv6. The following terminal output demonstrates all four implemented system calls running successfully:

- Demo 1 (getpinfo): Displays correct PID, PPID, State, and Process Name
- Demo 2 (pipe2 / IPC): Parent successfully sends message to child via pipe
- Demo 3 (thread_create): Thread executes concurrently with correct argument
- Demo 4 (mutex_lock/unlock): Shared counter increments safely with lock protection

```
=== xv6 Custom Syscall Demo ===

--- Demo 1: getpinfo (Process Info) ---
  PID:   3
  PPID:  2
  State: 4 (3=RUNNING)
  Name:  syscall_demo

--- Demo 2: pipe2 (IPC Pipe with Flags) ---
  pipe2 created: read_fd=3, write_fd=4
  Parent received: "Hello via pipe2!"

--- Demo 3: thread_create (Threads) ---
  fn=0x0000000000000000 stack=0x0000000000014000
  Thread created   [witThread] Hello frohm thread! arg = 42
  [Thread] My PID = 5
 TID=5
  Thread finished.
```

```
--- Demo 4: mutex_lock/unlock (Locks) ---
  Star ted [Lock Worker lo 1ck ]w coorker TID=6
unter = 1
  [Lock Worker 1] counter = 2
  [Lock Worker 1] counter = 3
  [Lock Worker 1] counter = 4
  [Lock Worker 1] counter = 5
  Star te [d loLocck Wok worker TIDrker 2] counter = 1
  [Lock =7
Worker 2] counter = 2
  [Lock Worker 2] counter = 3
  [Lock Worker 2] counter = 4
  [Lock Worker 2] counter = 5
  Final shared_counter = 0
```

9. Conclusion

This project successfully demonstrates the implementation of custom system calls in the xv6 operating system. It provides hands-on exposure to:

- Kernel-level programming
- Process management
- Inter-process communication
- Synchronization mechanisms

Overall, it deepens understanding of how operating systems manage resources and execute system-level services.

---

10. Future Improvements

- Implement true multithreading with shared memory support
- Add advanced IPC mechanisms like pipes and signals
- Modify xv6 scheduler for performance testing
- Implement deadlock detection mechanisms
- Add priority-based scheduling